

HT68FB550 USB Mouse Application

D/N: AN0699EN

Introduction

The HT68FB550 is a Flash Memory I/O with USB type 8-bit high performance RISC architecture microcontroller, which is designed for applications which require an USB interface. The device provides a range of resources including an integrated SPI and a USB SIE, making the development of the USB mouse more efficient and convenient.

This document will introduce how to use the HT68FB550 MCU to implement a USB mouse, which mainly introduces the use of CMOS sensor function and USB function related registers and operation procedures for user reference.

Functional Description

The USB mouse is an external HID (Human Interface Device) device for computer, which is used to indicate the XY coordinate location of a computer system, making the PC operation easy and fast. The mouse implemented in this document consists of six control buttons, two RGB LEDs for backlighting, an XY coordinate acquisition sensor and a scroll wheel encoder. The mouse uses a USB to communicate with the computer.

The buttons are controlled by using pin scanning. The RGB lighting effect is controlled by TM module PWM output. The XY coordinate acquisition sensor usually consists of an LED and a CMOS sensor. The sensor uses the light emitted from the LED to make optical measurements and get the continuous positional changes, and further processes the data to get the relative displacement values Δx and Δy . These values are transmitted to the MCU via SPI communication after processing. The scroll wheel encoder is centered on a rotary encoder switch. The scroll wheel end of the mouse must be plugged into the rotary encoder switch. When the mouse scroll wheel scrolls, the scroll wheel will drive the encoder switch and generates a pulse signal. The MCU reads the signal to determine the scroll wheel scrolling direction. When the MCU detects a change in the XY coordinates, scroll wheel or buttons, the MCU will save this data and upload it to the host computer when an IN interrupt occurs.

HT68FB550 Basic Features

HT68FB550 features:

- Flash Program Memory: 8K×16
- RAM Data Memory: 512×8
- USB 2.0 Full Speed compatible
- Up to 6 endpoints supported
- 3.3V LDO and internal UDP 1.5kΩ pull-high resistor to boost the pin voltage to 3.3V
- Internal 12MHz HIRC with 0.25% accuracy for all USB modes
- Integrated two 10-bit CTMs, a 10-bit STM and a 16-bit STM
- Dual SPI and I²C serial interfaces
- Support In System Programming function – ISP

USB Module

The USB namely Universal Serial Bus is a 4-wire bus that allows communication between PCs and different devices, which are divided into different categories and each category has a common behavior and protocol to provide similar functionality. Refer to the relevant standard documents for the USB communication protocol, which will not be explained here. The following section will describe the functions and development process related to the HT68FB550 USB module.

Register Description

The registers related to the HT68FB550 USB function are SYSC, USB_STAT, UINT, USC, USR, UCC, AWR, STLO, STLI, SIES, MISC, UFOEN, UFIEN, UFC0, UFC1 and FIFO_n(n=0~5).

Register Name	Bit							
	7	6	5	4	3	2	1	0
SYSC	CLK_ADJ	USBdis	RUBUS	—	—	HFV	—	—
USB_STAT	PS2_CKO	PS2_DAO	PS2_CKI	PS2_DAI	SE1	SE0	PU	ESD
UINT	—	—	EP5EN	EP4EN	EP3EN	EP2EN	EP1EN	EP0EN
USC	URD	SELPS2	PLL	SELUSB	RESUME	URST	RMWK	SUSP
USR	—	—	EP5F	EP4F	EP3F	EP2F	EP1F	EP0F
UCC	Rctrl	SYSCCLK	Fsys16MHZ	SUSP2	USBCKEN	EPS2	EPS1	EPS0
AWR	AD6	AD5	AD4	AD3	AD2	AD1	AD0	WKEN
STLO	—	—	STLO5	STLO4	STLO3	STLO2	STLO1	STLO0
STLI	—	—	STLI5	STLI4	STLI3	STLI2	STLI1	STLI0
SIES	NMI	CRCF	—	NAK	IN	OUT	ERR	ASET
MISC	LEN0	READY	SETCMD	E4ODF	E3IDF	CLEAR	TX	REQUEST
UFOEN	—	—	SETO5	SETO4	SETO3	SETO2	SETO1	DATAG
UFIEN	—	—	SETI5	SETI4	SETI3	SETI2	SETI1	FIFO_def
UFC0	E3FS1	E3FS0	E2FS1	E2FS0	E1FS1	E1FS0	—	—
UFC1	—	—	—	—	E5FS1	E5FS0	E4FS1	E4FS0
FIFO0	D7	D6	D5	D4	D3	D2	D1	D0
FIFO1	D7	D6	D5	D4	D3	D2	D1	D0
FIFO2	D7	D6	D5	D4	D3	D2	D1	D0
FIFO3	D7	D6	D5	D4	D3	D2	D1	D0
FIFO4	D7	D6	D5	D4	D3	D2	D1	D0
FIFO5	D7	D6	D5	D4	D3	D2	D1	D0

- SYSC: used to enable the USB SIE (Serial Interface Engine) function.
- USB_STAT: used for the USB bus noise indication and the UDP/UDN pull-high function control.
- UINT: used for the USB endpoint interrupt enable control.
- USC: used for the USB or PS2 communication mode selection and USB status indication.
- USR: used to indicate the endpoint being accessed by the host.
- UCC: used for the UDP pull-high resistor selection, MCU clock source and USB clock enable control and endpoint FIFO access selection.
- AWR: used for the USB device address setting and USB remote wake-up enable control.
- STLO, STLI: used to indicate whether an endpoint is in a stall state.
- SIES: used to indicate the USB communication status.
- MISC: used for the FIFO read/write operations.
- UFIEN, UFOEN: used for the endpoint input/output function enable and disable control.
- UFC0, UFC1: used to set the endpoint FIFO size.
- FIFOn: used to store data to be sent or read by the endpoint.

Refer to the HT68FB550 Datasheet for detailed register information.

Endpoint Description

The HT68FB550 supports six endpoints: endpoint 0 ~ endpoint 5. The endpoint 0 supports control transfer with a FIFO buffer size of 8 bytes. The endpoint 1 ~ endpoint 5 support interrupt and bulk transfer, and their FIFO buffer can be configured to 8, 16, 32, or 64 bytes.

In this example, the endpoint 0 and endpoint 1 are used for data transfer.

- The endpoint 0 is used for control transfer between the mouse and the host. It contains operations such as device initialisation, descriptor upload, etc. The FIFO buffer size is 8 bytes.
- The endpoint 1 performs data transmission of the mouse device by using interrupt transfer. The host accesses the mouse state data about movement, scrolling and button clicking states every 1ms interval. The FIFO buffer size is 8 bytes. The RAM buffer creates an array, which contains the 5 button values (except the DPI button), the sensor X, Y relative coordinate values, and the incremental value of the scroll wheel scrolling. The RAM buffer is modified immediately each time the mouse state changes. When the endpoint 1 interrupt is triggered, upload the RAM buffer data, clear the buffer, and then repeat the preceding steps.

Steps to Develop a USB Mouse Interface

1. USB initialisation

- (1) Configure the UCC register: set the Fsys16MHZ bit high to select the system clock source from the PLL output 16MHz, and set the USBCKEN bit high to enable the USB clock.
- (2) Configure the SYSC register to enable the USB SIE.
- (3) Configure the USC register: set the URD bit high to allow the USB reset signal to reset the MCU. Set the SELUSB bit to enable the USB mode.

- (4) Configure the UFC0 and UFC1 registers to configure the FIFO size for the corresponding endpoints.
- (5) Configure the UFIEN register to enable the corresponding endpoint FIFO input.
- (6) Configure the INTC0 register to enable the USB interrupt.
- (7) Configure the UINT register to enable the corresponding USB endpoint interrupts.
2. Set the device descriptor, configuration descriptor, endpoint descriptor, interface descriptor and report descriptor of the device according to the USB category.
3. Execute the enumeration program in the endpoint 0 interrupt, receive the request from the host, and upload the corresponding descriptor.
4. Create a report buffer with the same data structure as the report descriptor to store the uploaded reports.
5. Fill the report buffer and set the upload report flag.
6. Determine whether to upload the report in the current report buffer based on the IN interrupt request from other endpoints and the upload report flag.

CMOS Sensor Development Description

In this application note, the ADNS-3050 is used as a CMOS Sensor. The moving directional information is obtained through the CMOS sensor and transmitted to the MCU via SPI communication.

ADNS-3050 Introduction

The Avago Technologies ADNS-3050 is a small form factor entry-level gaming optical navigation sensor. It is housed in an 8-pin staggered dual in-line package (DIP). The sensor has the following features.

- High-speed motion detection: high-speed motion detection on white paper at a typical speed of 160IPS with an acceleration up to 23G. Usually, high-speed motion detection at a speed of 60IPS with an acceleration up to 20G
- Selectable resolutions up to 2000DPI
- 4-wire serial interface, easy to communicate with MCU
- External interrupt output for motion detection
- Internal oscillator — no clock input needed
- On-chip LED driver
- Minimal number of passive components
- Programmable power-saving modes for selectable wired or wireless application
- Can be switched to the sleep mode
- Configurable LED operating modes and drive current

HT68FB550 SPI Interface Description

The HT68FB550 device contains an independent Serial Peripheral Interface function (SPIA). The SPI interface is often used to communicate with external peripheral devices such as sensors, Flash or EEPROM memory devices after a simple configuration. The following section will introduce the registers related to the HT68FB550 SPIA module. Refer to the HT68FB550 Datasheet for the detailed functional description.

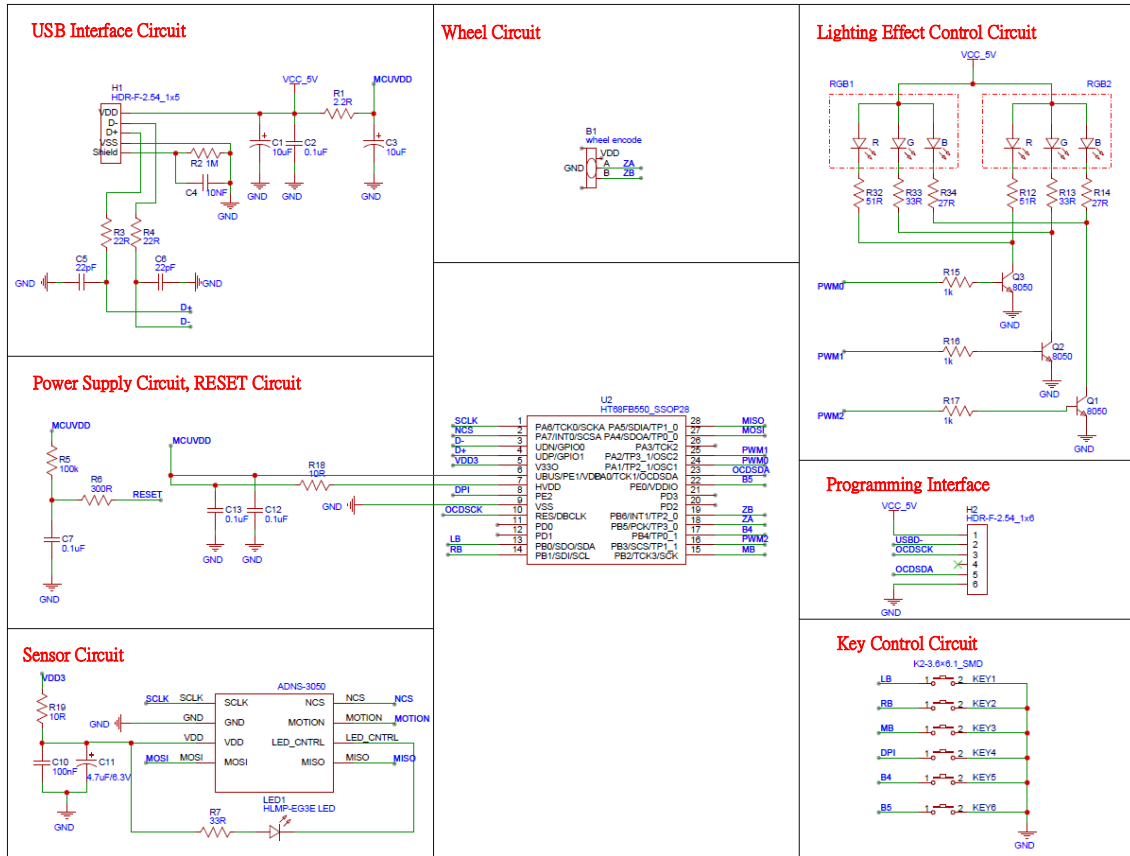
Register Name	Bit							
	7	6	5	4	3	2	1	0
SPIAC0	SASPI2	SASPI1	SASPI0	—	—	—	SPIAEN	—
SPIAC1	—	—	SACKPOL	SACKEG	SAMLS	SACSEN	SAWCOL	SATRF
SPIAD	D7	D6	D5	D4	D3	D2	D1	D0
SBSC	SIM_WCOL	—	I2CDB1	I2CDB0	—	—	—	SA_WCOL

- SPIAC0~SPIAC1: used to control the SPIA master/slave selection and the SPIA master clock frequency, the SPIA enable control bit, data capture on the SCKA rising or falling edge.
- SPIAD: used to store the data being transmitted and received.
- SBSC: used to control the SPIA WCOL function.

CMOS Sensor Development Steps

1. Set the SACSEN and SAMLS bits to enable the $\overline{\text{SCSA}}$ pin function and select MSB first. Disable the SPIA write collision.
2. Set the SPIAC0 control register to set the SPI to master mode with a clock of $f_{\text{sys}}/64$.
3. Set the SPIAEN enable control bit to enable the SPI function.
4. Write a reset command to the CMOS sensor corresponding address via SPI communication to reset and initialise the CMOS sensor.
5. Initialise the CMOS sensor DPI by writing a DPI value to the corresponding address of the CMOS Sensor via SPI communication.
6. When the MCU detects a change in the XY coordinate position, it will read the address where the CMOS sensor stores the $\Delta X/\Delta Y$ coordinates via SPI communication to obtain the $\Delta X/\Delta Y$ coordinates.
7. Accumulate the ΔX coordinate value read each time, invert the ΔY coordinate value read each time and add 1, and finally store the accumulated results into the buffer respectively. When the interrupt is triggered, send the data in the buffer to PC, then clear the accumulated results, and so on.

Hardware Circuit Description



USB interface circuit: the circuit mainly provides a power supply for the whole mouse and implements the communication with the USB host. The power supply provided by the USB host is filtered by the C1 and C2 capacitors to provide a stable power supply for the system. The MCU connects to the UDP pin through an integrated 1.5kΩ resistor without requiring additional external pull-high resistor. The resistors and capacitors of the USB signal lines are mainly for impedance matching for the signal lines to reduce the interference and improve the communication reliability, and thus reduce the probability of data error during USB data transmission.

Sensor circuit: the MCU integrated LDO can directly output a 3.3V voltage on the V330 pin for the ADNS-3050 sensor. The circuit eliminates the requirement for an external LDO, thus saving component costs. The ADNS-3050 sensor communicates with the MCU integrated 4-wire SPI, thus simplifying the communication process.

Lighting effect control circuit: the four MCU integrated Timer Modules (TM) are used for LED control. Three TMs are used to generate PWM signals to drive three transistors, thus driving RGB LEDs. In this example, two RGB LEDs are used as the backlight, and various colors can be implemented by controlling the duty cycle of the three PWM outputs. The remaining one TM is used to complete the breathing effect of fading out and brightening up and the timing function.

Button control circuit: the Left Button (LB), Right Button (RB), Middle Button (MB), Forward Button (B4), Backward Button (B5) and DPI Button are respectively connected to the PB0, PB1, PB2, PB4, PE0 and PE2 pins of the MCU. These pins are configured as inputs with their internal

pull-high function enabled. When the pin level has been detected as low by the button scan, it means that the button has been pressed.

Scroll wheel circuit: the scroll wheel encoder signal pin A and pin B are respectively connected to the MCU PB5 and PB6 pins. These pins are configured as inputs with their internal pull-high function enabled. According to the principle of the encoder, when the scroll wheel rotates clockwise, the pin levels change in an order of 11→01→00→10. When the scroll wheel rotates counterclockwise, the pin levels change in an order of 11→10→00→01. The MCU can know the current level signal state by scanning and reading the I/O level changes of the two pins, and compare it with the next level signal state to determine whether the scroll wheel scrolls up or down at this time. For example, if the current level signal state is 0b11 and the next level signal state is 0b01, it will scroll up.

Software Description

Use a simulation development tool, the HT-IDE3000, to open the program example project which is provided in this application note. Users can select the system-defined configuration options on the HT-IDE3000 during programming, and the specific configuration options are as follows:

V_{DD} (for selections of Internal RC): 5V
 High Freq. OSC: HIRC
 I/O(PE0) or VDDIO Function: I/O(PE0)

The following sections will describe the used program examples in terms of project documentation, descriptors, and software flowcharts.

Project File Description

The main project files and their descriptions are listed in the following table.

File Name	Description
USBGamingMouse.c	Main program file, which contains the main loop and initialisation code
Mouse.c	Used for sensor related operations and scroll wheel scrolling checks
POCESSBUTTON.c	Used for button check and processing of button and scroll wheel functions
PulsationPWM.c	Used to implement the function of lighting effects
Timer.c	Used to control the timing function of the light
profile.c	Used to configure the constant values for lighting effects and DPI, etc.
IAPProgram.C	Used for storage and modification of Flash data
USBModule.c	Used for USB related operations, including descriptor definition, USB communication, etc.
Variable.c	Definitions of global variables for the entire program

Descriptors

The USB mouse belongs to USB HID (Human Interface Device) class. The key to implement the USB mouse functions using the HT68FB550 is to set various descriptors, which can describe the device as the user-required device. The host can know that the USB mouse is an HID device according to the descriptors, and make corresponding actions to communicate with it. The following section will describe the various descriptors of the USB mouse in the program example.

Device Descriptor

The device descriptor is used to describe information such as the device class, the protocol used by the device, the device manufacturer, the product, and so on.

Field	Position	Size/Byte	Name	Example
bLength	0	1	Descriptor length	0x12
bDescriptorType	1	1	Descriptor type	0x01
bcdUSB	2	2	Protocol version	0x0200
bDeviceClass	4	1	Device class	0x00
bDeviceSubClass	5	1	Device subclass	0x00
bDeviceProtocol	6	1	Device protocol code	0x00
bMaxPckSize0	7	1	Maximum packet size	0x08
idVendor	8	2	Manufacturer identifier	0x04D9
idProduct	10	2	Product identifier	0x0B00
bcdDevice	12	2	Device version	0x0001
iManufacturer	14	1	Manufacturer name index	0x01
iProduct	15	1	Product name index	0x02
iSerialNumber	16	1	Product serial number index	0x00
bNumConfigurations	17	1	Number of configurations	0x01

- bLength: indicates the length of the current device descriptor. In this example, the bLength is 0x12 in hexadecimal, which is 18 in decimal, indicating that the length of the device descriptor is 18.
- bDescriptorType: indicates the descriptor type. The specific value is based on the USB protocol. The value for the device descriptor type is fixed to 0x01.
- bcdUSB: indicates the USB protocol version of the current device. In this example, 0x0200 indicates USB v2.0, and the common one is 0x0110, which indicates USB v1.1.
- bDeviceClass: indicates the device class, as specified by the USB protocol. The general USB device class code field is set to 0. When bDeviceClass is 0, bDeviceSubClass used below is also required to be 0. If bDeviceClass is set to 0xFF, it indicates that the device is a vendor-defined device class.
- bDeviceSubClass: indicates the subclass code used by the device, as specified by the USB protocol. When bDeviceClass is set to 0, this field must also be set to 0.
- bDeviceProtocol: indicates the protocol used by the device, as specified by the USB protocol. When bDeviceClass is set to 0, this field must also be set to 0.
- bMaxPckSize0: indicates the maximum packet length of endpoint 0.
- idVendor: indicates the vendor ID number. The value is assigned by the USB association.
- idProduct: indicates the product ID, and can be customized by the manufacturer.
- bcdDevice: indicates the device version number, and can be customized by the manufacturer.
- iManufacturer: indicates the index value of the vendor string. When the host requires to get the vendor string, the host will set the LSB of wValue to this index value to get the vendor string.
- iProduct: indicates the index value of the product string. When the host requires to get the product string, the host will set the LSB of wValue to this index value to get the product string.
- iSerialNumber: indicates the index value of the device serial number string. It is used to distinguish different devices for the host. In this example, it is set to 0, it means there is no serial number string.

- **bNumConfigurations:** indicates how many configurations the device has, each configuration has its own configuration descriptor, and the host selects the device configuration using the configuration setting command. In this example, it is set to 1, indicating that there is only one configuration.

Configuration Descriptor

A complete set of configuration descriptors includes standard configuration descriptors, interface descriptors and endpoint descriptors. If the device is an HID device, the HID descriptor is also included in the configuration descriptor set. The complete set of configuration descriptors in this example is described below.

- **Standard Configuration Descriptor**

The standard configuration descriptor is used to indicate information such as the number of interfaces that this configuration can support, the index value of this configuration string, the value of this configuration, and the current required by the device.

Field	Position	Size/Byte	Name	Example
bLength	0	1	Descriptor length	0x09
bDescriptorType	1	1	Descriptor type	0x02
bTotalLength	2	2	Total length of configuration set	0x0022
bNumInterfaces	4	1	Number of interfaces	0x01
bConfigurationValue	5	1	Configuration value	0x01
iConfiguration	6	1	Configure string index	0x00
bmAttributes	7	1	Device configuration attributes	0xa0
bMaxPower	8	1	Maximum current consumption	0x32

- **bLength:** indicates the length of the current standard descriptor. In this example, the bLength is 0x09 in hexadecimal, which is 9 in decimal, indicating that the length of the device standard configuration descriptor is 9.
- **bDescriptorType:** indicates the descriptor type. The specific value is based on the USB protocol. The value of the configuration descriptor type is fixed to 0x02.
- **bTotalLength:** indicates the total length of the configuration descriptor set, including interface descriptors, endpoint descriptors, and HID descriptors. The 0x0022 in hexadecimal means that this configuration descriptor set has 34 bytes.
- **bNumInterfaces:** indicates the number of interfaces supported by this configuration descriptor. This example is a mouse device, so only one interface is required.
- **bConfigurationValue:** indicates the configuration value. Generally one device supports multiple configurations, and the host selects the desired device configuration to enable based on the wValue LSB value of the configuration setting command.
- **iConfiguration:** indicates the index value of this configuration string. In this example, no configuration string is set, so the value is 0.
- **bmAttributes:** indicates some features of the device. As specified by the USB protocol, D7 in this field is reserved and must be set to 1. D6 indicates the power supply mode. When it is 1, it means that the device is self-powered. When the value is 0, the device is powered by the bus. D5 indicates whether the remote wake-up is supported, and when D5 is set to 1, the

remote wake-up is supported. D4~D0 are reserved and set to 0. In this example, the bmAttributes is set as follows.

D7	D6	D5	D4	D3	D2	D1	D0
1	0	1	0	0	0	0	0

0xA0 means that the USB device is powered by bus and supports remote wake-up.

- bMaxPower: indicates the device maximum current consumption in units of 2mA. In this example, the device maximum current consumption is $50 \times 2 = 100\text{mA}$.

● Interface Descriptor

An interface descriptor is used to describe an interface in a configuration. Each interface descriptor has information about the class, subclass, and protocol. These information can be used to specify the current device class when the bDeviceClass is set to 0 in the device descriptor.

Field	Position	Size/Byte	Name	Example
bLength	0	1	Descriptor length	0x09
bDescriptorType	1	1	Descriptor type	0x04
bInterfaceNumber	2	1	Interface number	0x00
bAlternateSetting	3	1	Interface alternate number	0x00
bNumEndpoints	4	1	Number of endpoints	0x01
bInterfaceClass	5	1	Interface class	0x03
bInterfaceSubClass	6	1	Interface subclass	0x01
bInterfaceProtocol	7	1	Interface protocol code	0x02
iInterface	8	1	Interface string index	0x00

- bLength: indicates the length of the current interface descriptor. In this example, the bLength is 0x09 in hexadecimal, which is 9 in decimal, indicating that the length of the device descriptor is 9.
- bDescriptorType: indicates the descriptor type. The specific value is based on the USB protocol. The value for the interface descriptor type is fixed to 0x04.
- bInterfaceNumber: indicates the interface number. When more than one interface exists, each interface has a different number. The interfaces are numbered starting from 0.
- bAlternateSetting: indicates the alternate number of the interface.
- bNumEndpoints: the number of endpoints used by the interface, excluding endpoint 0. In this example, only one endpoint is used.
- bInterfaceClass, bInterfaceSubClass, bInterfaceProtocol: indicates the interface class, interface subclass, and interface protocol. In the current application note, the device descriptor bDeviceClass is set to 0, indicating that these three fields are used to specify the USB device class. In this example, the bInterfaceClass is set to 0x03, it means that the interface belongs to the HID class.
- iInterface: the index value of the interface string. There is no interface string to set in this example, so the iInterface is set to 0.

● HID Descriptor

The HID descriptor is used to describe the number, length, and type of the device sub-descriptor, which is generally a report descriptor.

Field	Position	Size/Byte	Name	Example
bLength	0	1	Descriptor length	0x09
bDescriptorType	1	1	Descriptor type	0x21
bcdHID	2	2	HID protocol code	0x0110
bCountryCode	4	1	Country code	0x00
bNumDescriptors	5	1	Number of sub-descriptors	0x01
bDescriptorType	6	1	Sub-descriptor type	0x22
wDescriptorLength	7	2	Sub-descriptor length	0x0064

- bLength: indicates the length of the current HID descriptor. In this example, the bLength is 0x09 in hexadecimal, which is 9 in decimal, indicating that the length of the device HID descriptor is 9.
- bDescriptorType: indicates the descriptor type. The specific value is based on the USB protocol. The value for the HID descriptor type is fixed to 0x21.
- bcdHID: indicates the HID protocol version number of the device. In this example, the bcdHID is set to 0x0110, so the mouse device in this example uses the USB HID1.1 protocol.
- bCountryCode: indicates the country where the device is suitable for use.
- bNumDescriptors: indicates the number of sub-descriptors. In this example, there is only one sub-descriptor, so the bNumDescriptors is set to 0x01.
- bDescriptorType: indicates the type of sub-descriptor. The sub-descriptor in this example is the report descriptor. Therefore, the bDescriptorType is set to 0x22 according to the USB protocol.
- wDescriptorLength: indicates the length of the sub-descriptor. In this example, the report descriptor has only 100 bytes, so the field is 0x0064.

Note: When there are multiple sub-descriptors, the definitions of the bDescriptorType and wDescriptorLength can be repeated alternately, and so on.

● Endpoint Descriptor

An endpoint descriptor is used to describe the endpoint function used in device communication.

Field	Position	Size/Byte	Name	Example
bLength	0	1	Descriptor length	0x07
bDescriptorType	1	1	Descriptor type	0x05
bEndpointAddress	2	1	Endpoint address	0x81
bmAttributes	3	1	Endpoint attribute	0x03
wMaxPacketSize	4	2	Endpoint maximum packet size	0x0008
bInterval	6	1	Endpoint polling time	0x01

- bLength: indicates the length of the current endpoint descriptor. In this example, the bLength is 0x07 in hexadecimal, which is 7 in decimal, indicating that the length of the device endpoint descriptor is 7.
- bDescriptorType: indicates the descriptor type. The specific value is based on the USB protocol. The value for the endpoint descriptor type is fixed to 0x05.

- bEndpointAddress: indicates the endpoint address. Here D7 indicates the transmission direction of the endpoint, 1 is input and 0 is output. D3 ~ D0 indicate the endpoint number; D6~D4 are reserved and set to 0. The bEndpointAddress in this example is set as follows.

D7	D6	D5	D4	D3	D2	D1	D0
1	0	0	0	0	0	0	1

0x81 means that the transmission direction is input and the endpoint number is 1.

- bmAttributes: a value of 0x03 indicates that this endpoint is used for interrupt transfer.
- wMaxPacketSize: indicates the maximum packet length that can be transmitted by this endpoint.
- bInterval: indicates the endpoint polling time. It indicates that the time interval for the host to poll the endpoint with an interrupt is 1ms.

● Report Descriptor

```

0x05,0x01,          //Usage Pg(Generic Desktop)=05H,01H
0x09,0x02,          //Usage (Mouse)=09H,02H
0xa1,0x01,          //Collection: (Application)=A1H,01H
0x85,MOUSE_REPORT_ID, //Report ID=85 01
0x09,0x01,          //Usage (Pointer)=09H,01H
0xa1,0x00,          //Collection (Physical)=A1H,00H
0x05,0x09,          //Usage Page (Buttons)=09H,05H
0x15,0x00,          //Logical Minimum(0)=15H,00H
0x25,0x01,          //Logical Maximum(1)=25H,01H
0x19,0x01,          //Usage Minimum=19H,01H
0x29,0x05,          //Usage Maximun=29H,05H
0x75,0x01,          //Report Size=75H,01H
0x95,0x05,          //Report Count=95H,05H
0x81,0x02,          //Input (Data, Variable, Absolute)=81H,02H
0x95,0x03,          //Report Count=95H,03H
0x81,0x01,          //Input (Constant)=81H,01H
0x05,0x01,          //Usage Page (Generic Desktop)=05H,01H
0x15,0x81,          //Logical Minimum (-127)=16H,0x081
0x25,0x7f,          //Logical Maximum (127)=26H,0x07f
0x09,0x30,          //Usage (X)=09H,30H
0x09,0x31,          //Usage (Y)=09H,31H
0x75,0x08,          //Report Size (16)=75H,08H
0x95,0x02,          //Report Count (2)=95H,02H
0x81,0x06,          //Input (Data, Variable, Relative)=81H,06H
0x15,0x81,          //Logical Minimum (-127)=15H,81H
0x25,0x7f,          //Logical Maximum (+127)=25H,7FH

```

```

0x09,0x38, //Usage (Wheel)=09H,38H
0x75,0x08, //Report Size (8)=75H, 8H
0x95,0x01, //Report Count (1)=95H,01H
0x81,0x06, //Input (Data, Variable, Relative)=81H,06H
0x05,0x0c, //USAGE_PAGE (Consumer Devices)
0x0a,0x38,0x02, //AC Pan
0x95,0x01, //Report Count (1)=95H,01H
0x81,0x06, //Input (Data, Variable, Relative)=81H,06H
0xc0, //End Collection=C0H,C0H

```

Byte	D7	D6	D5	D4	D3	D2	D1	D0
0	Report ID							
1	Reserve			B5	B4	RB	MB	LB
2	X							
3	Y							
4	Scroll wheel							

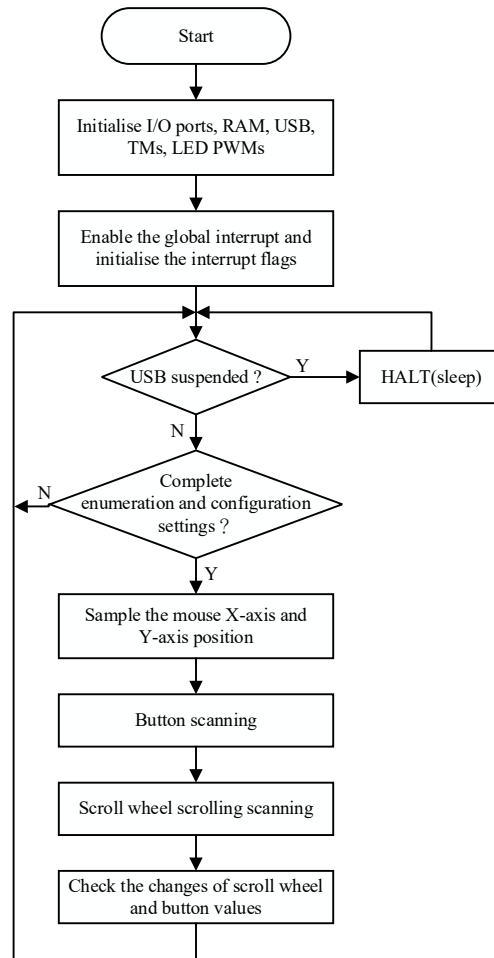
Note: LB (Left Button), MB (Middle Button), RB (Right Button), B4 (Button 4), B5 (Button 5).

Software Flowchart

When the device descriptor has been set, a USB enumeration is required for the device to upload the set the descriptor to the host to inform the host that the device is a mouse device. When the enumeration has completed, the device will check the relevant flag and determine whether to send the X-axis position and Y-axis position sampled by the CMOS sensor, the 5 button press information (except DPI button), and the scroll wheel scrolling information to the host.

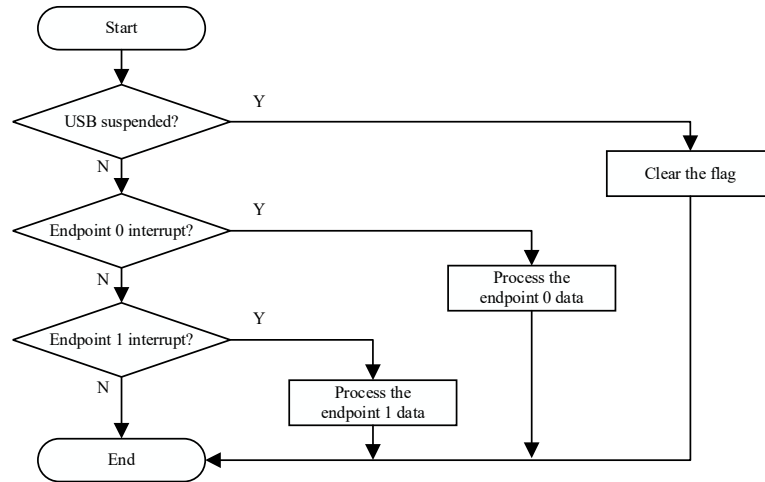
The overall program structure can be divided into: main program, USB interrupt subroutine, endpoint FIFO read/write subroutine, endpoint 0 data processing subroutine, endpoint 1 data processing subroutine, mouse X-axis and Y-axis position sampling subroutine, button scanning subroutine, scroll wheel scrolling scanning subroutine, and scroll wheel and button value checking subroutine.

Main Program



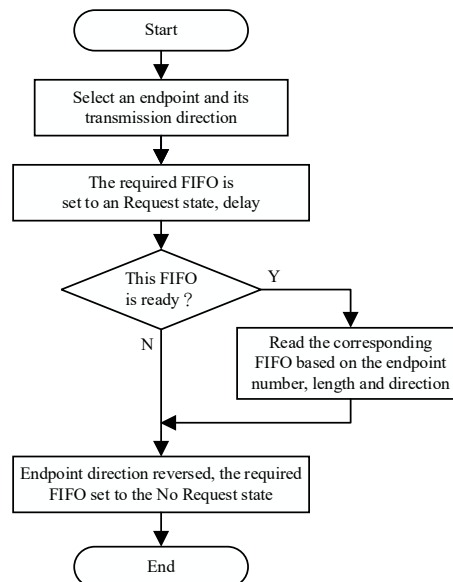
1. Initialise the I/O ports, RAM, USB configurations, LED PWMs and Timers.
2. Enable the global interrupt and clear variable flags.
3. Determine whether the USB module is suspended, if not, the program will go on. If yes, execute the HALT command to enter sleep mode, and the program will circularly determine whether the USB is suspended.
4. Determine whether the USB has completed the enumeration and configurations, if not, wait for the enumeration and configuration settings to complete.
5. When the enumeration and configurations have been completed, sample the mouse X-axis and Y-axis position, scan the button and scroll wheel scrolling and check the scroll wheel and button value changes, and finally go to Step 3 to start a new loop.

USB Interrupt Subroutine



1. When an endpoint is accessed, the USB interrupt request flag USBF will be set high and an USB interrupt request will be generated.
2. Determine whether the USB module is suspended, if not, the program will go on. If yes, clear the flag and end the interrupt service subroutine.
3. Determine which endpoint was accessed to trigger this interrupt and execute the corresponding endpoint processing subroutine.
 - (1) Endpoint 0: used for enumeration.
 - (2) Endpoint 1: used to upload the mouse related data (such as buttons, scroll wheel, and X-axis and Y-axis relative coordinate values).

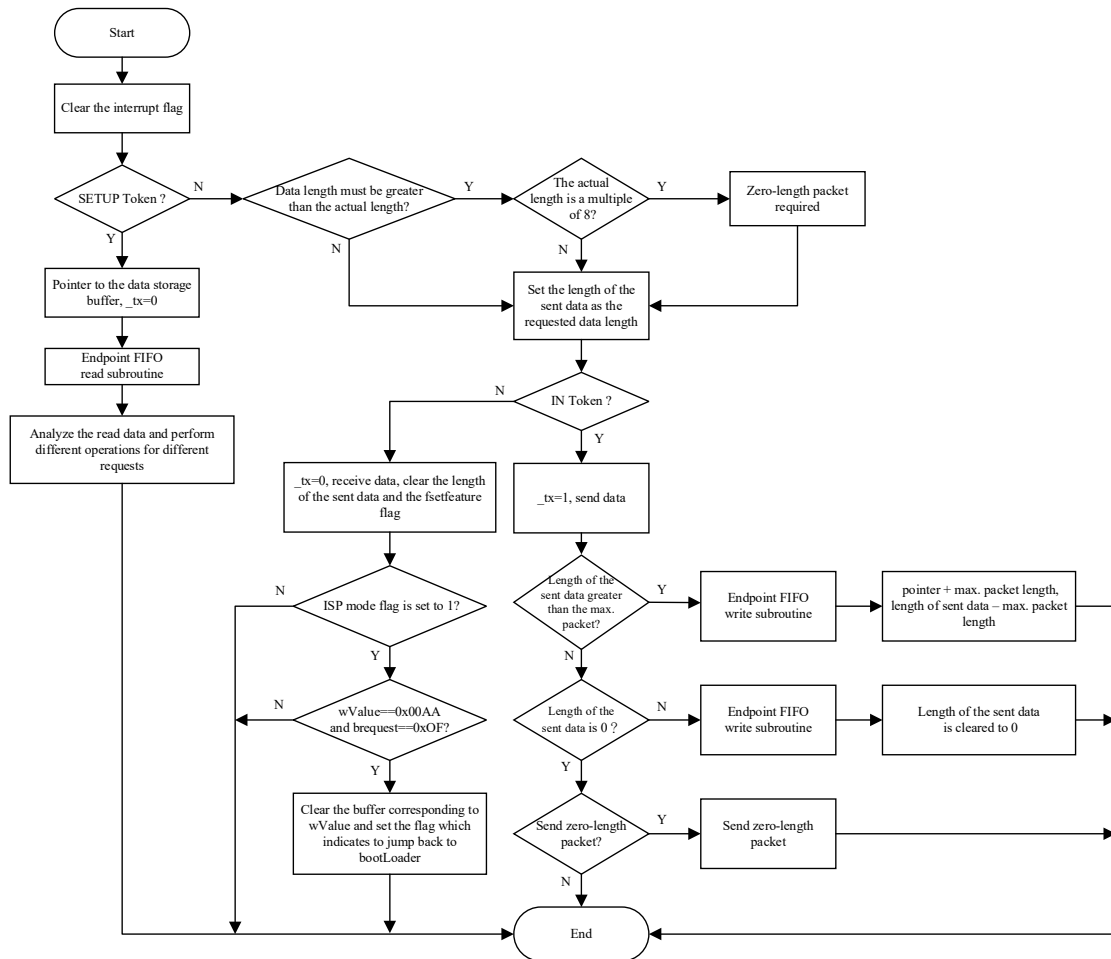
Endpoint FIFO Read/Write Subroutine



1. Set the `_ucc` (UCC register) to select an endpoint and set the `_tx` (the TX bit in the MISC register) to specify its transmission direction.

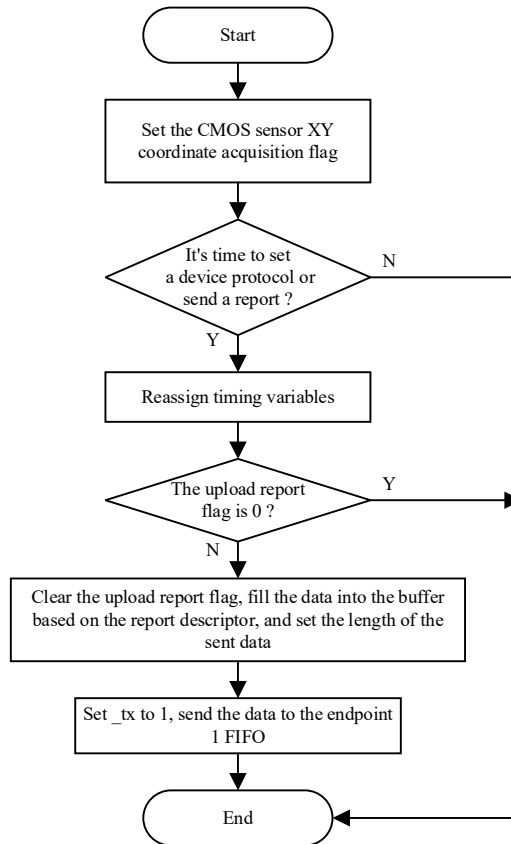
2. Select the endpoint and set the `_request` (the REQUEST bit in the MISC register) to 1 for a request to access to the corresponding FIFO.
3. Determine whether the required FIFO is in a ready state. If it is in a ready state, users can read and write the corresponding FIFO according to the endpoint number, length and other information.
4. After the FIFO read/write operation has completed, reverse the `_tx` bit and set the FIFO to the No Request state by clearing the `_request` bit.

Endpoint 0 Data Processing Subroutine



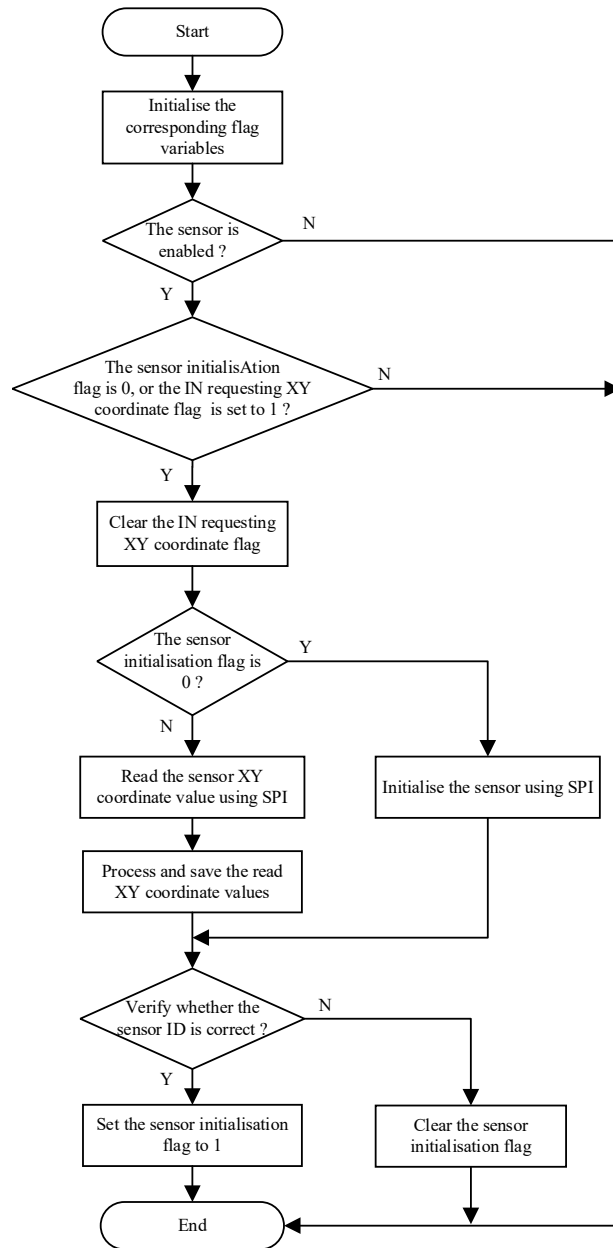
1. Determine whether this interrupt is triggered due to a SETUP request. If yes, read the data and parse the data packet for the SETUP request, and then perform different operations such as data loading and flag settings based on the parsed results.
2. Determine whether this interrupt is triggered due to an IN request, if yes, set the `_tx` bit to 1 and send the data.
3. If the interrupt is not triggered by an IN request, set the `_tx` bit to 0 and read the data, then determine whether the ISP mode flag is set to 1. If yes, determine whether the states of `wValue` and `bRequest` in the read data are correct, and jump to the `bootLoader` to 1 if their states are correct.

Endpoint 1 Data Processing Subroutine



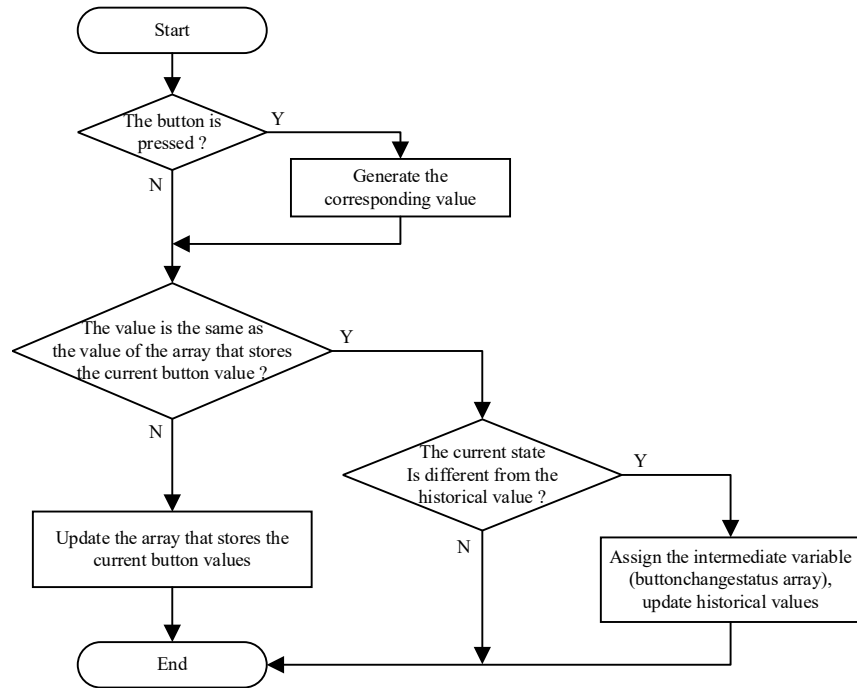
1. Set the CMOS sensor XY coordinate acquisition flag so that the XY coordinate information can be acquired in the main function.
2. Determine whether it is time to upload the report by checking the report rate counter decrement situation. Exit the interrupt if the time is not reached, or the program goes on when the time is reached.
3. Reassign the timing variables.
4. If the fsendmousecode flag is set to 1, clear the upload report flag, fill the data into the buffer according to the report descriptor format, and set the length of the sent data.
5. Set the _tx bit to 1, upload report to the host via the endpoint 1 FIFO, then end the interrupt service subroutine.

Mouse X-axis and Y-axis Position Sampling Subroutine



1. When the main function has completed the initialisation, the sensor enable flag will be set to 1.
2. Determine whether the sensor is enabled according to the sensor enable flag, if yes, the program will go on.
3. Determine whether the sensor is initialised or whether the sensor request flag is set to 1. If the sensor is not initialised, initialise the sensor. If the sensor has been initialised, read the XY coordinate data.
4. Read the sensor ID, if the ID is matched, set the sensor initialisation flag to 1, otherwise it will be cleared.

Button Scanning Subroutine

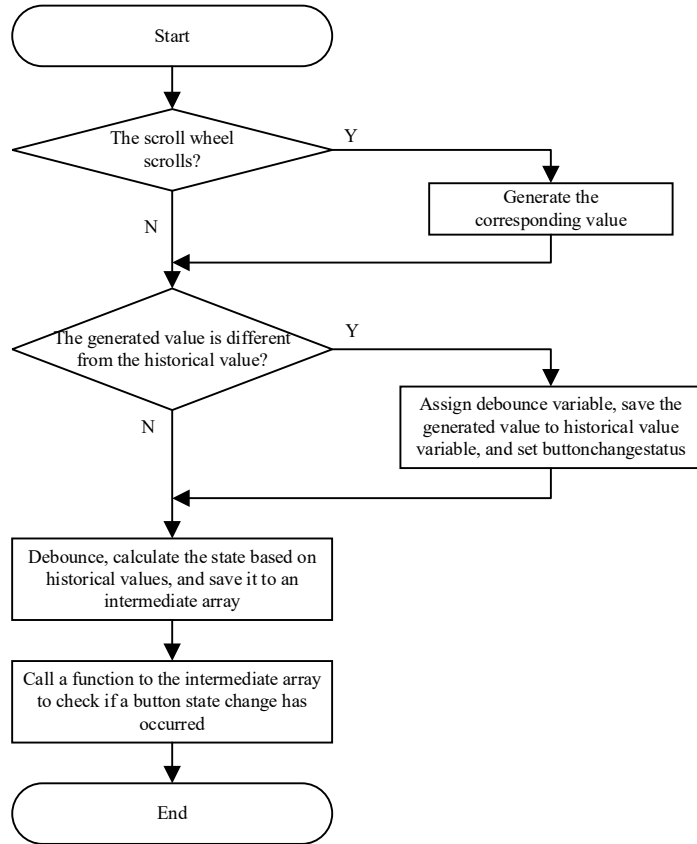


1. Determine whether a button is pressed. If the button is pressed, the corresponding value will be generated, and the available button values are described in the following table.

Button Type	Button Value
Left Button	0b00000001
Right Button	0b00000010
Middle Button	0b00000100
DPI Button	0b00001000
Forward Button	0b00010000
Backward Button	0b00100000

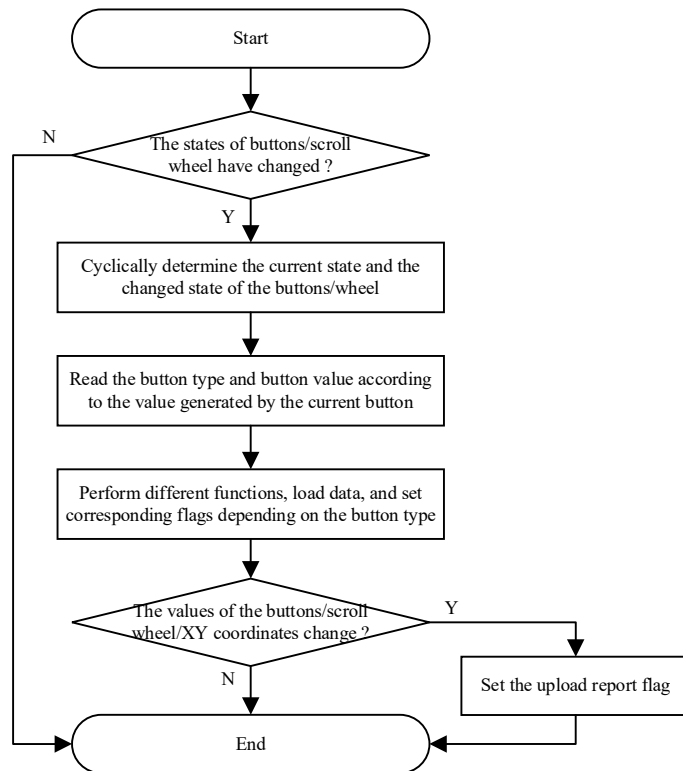
2. Compare the value generated by the currently pressed button with the buttoncurrentstatus array, if different, assign the value generated by the currently pressed button to the buttoncurrentstatus array and end the subroutine.
3. If the comparison result in Step 2 is the same, wait for a debounce, then use the buttoncurrentstatus array and the buttonpreviousstatus array to make an XOR judgment. If the current state is different from the previous state, assign the buttonchangestatus array, end the scan and wait for the subsequent function to process.

Scroll Wheel Scrolling Scanning Subroutine



1. Read the I/O port level states of the scroll wheel to determine whether the scroll wheel scrolls. If the scroll wheel scrolls, the corresponding value will be generated.
2. Perform the XOR judgment based on the generated value and historical value. If the two values are different, save the generated value to the historical value variable and assign the debounce variable.
3. When the debounce has completed, calculate the state value based on the historical value and save it to the intermediate state array, buttonchangestatus.
4. Determine whether a button value change has occurred based on the intermediate state array, and set the flag high if the change occurs.

Scroll Wheel and Button Value Checking Subroutine



1. Determine whether the state of the button and scroll wheel has changed according to the buttonchangestatus array mentioned in the scroll wheel and button scanning subroutine flowchart. If yes, the program will go on.
2. Determine the current state again according to the buttonchangestatus array. If the state is the same, the program goes on.
3. Read the button type and button value based on the current button state.
4. Perform different functions according to the button value type, load data, and set the corresponding flags in the function.
5. Determine whether all mouse data has changed according to multiple flags, and if yes, set the upload report flag to 1.

Conclusion

The application note has taken the HT68FB550 as the main control MCU and explained how to use its integrated SPI and USB functions to implement a USB mouse with six control buttons, two RGB LEDs for backlighting, an XY coordinate acquisition sensor and a scroll wheel encoder. The following steps need to be accomplished to develop a USB mouse.

1. Set a descriptor for the device to ensure that the host can correctly identify the device.
2. Complete the enumeration (i.e., inform the host of the device type and functions so that the host can configure and use the device correctly).
3. Perform the button, scroll wheel and XY coordinate scanning. Set the relevant flag if their states change.
4. Get the latest data to be uploaded and fill it in the report buffer.
5. In the IN request interrupt, determine whether the data in the report buffer needs to be uploaded based on the upload report flag.

Hopefully, the above instructions will be of some help to readers and provide a reference for the development of similar projects.

Reference Material

Reference file: HT68FB550 Datasheet

For more information, refer to the Holtek official website <https://www.holtek.com>.

Version and Modification Information

Date	Author	Issue	Modification Information
2024.11.13	王冠中/勞一峰	V1.00	First version

Disclaimer

All information, trademarks, logos, graphics, videos, audio clips, links and other items appearing on this website ('Information') are for reference only and is subject to change at any time without prior notice and at the discretion of Holtek Semiconductor Inc. and its related companies (herein after 'Holtek', 'the company', 'us', 'we' or 'our'). Whilst Holtek endeavors to ensure the accuracy of the Information on this website, no express or implied warranty is given by Holtek to the accuracy of the Information. Holtek shall bear no responsibility for any incorrectness or leakage.

Holtek shall not be liable for any damages (including but not limited to computer virus, system problems or data loss) whatsoever arising in using or in connection with the use of this website by any party. There may be links in this area, which allow you to visit the websites of other companies. These websites are not controlled by Holtek. Holtek will bear no responsibility and no guarantee to whatsoever Information displayed at such sites. Hyperlinks to other websites are at your own risk.

Limitation of Liability

In no event shall Holtek Limited be liable to any other party for any loss or damage whatsoever or howsoever caused directly or indirectly in connection with your access to or use of this website, the content thereon or any goods, materials or services.

Governing Law

The Disclaimer contained in the website shall be governed by and interpreted in accordance with the laws of the Republic of China. Users will submit to the non-exclusive jurisdiction of the Republic of China courts.

Update of Disclaimer

Holtek reserves the right to update the Disclaimer at any time with or without prior notice, all changes are effective immediately upon posting to the website.